CORPORACIÓN UNIVERSITARIA REMINGTON

Evaluación parcial número 2 de Lenguaje de Programación Avanzado 2

Implementar las siguientes funcionalidades sobre el proyecto **medicitas** que se entrega como insumo.

1. El usuario necesita consultar las citas médicas con estos filtros opcionales:

- Estado: PROGRAMADA, ATENDIDA o CANCELADA.
- Médico: buscar por el ID del médico.
- Paciente: buscar por el ID del paciente.
- Especialidad: buscar por especialidad médica.
- Fecha inicial y fecha final: consultar citas dentro de un rango.
- Nombre del paciente: buscar por coincidencia parcial, sin distinguir mayúsculas y minúsculas.

Por ejemplo, una petición podría ser:

```
GET /api/citas?estado=PROGRAMADA&medicoId=101&fechaDesde=2026-11-01T00:00:00
    &fechaHasta=2026-11-30T23:59:59
```

O… también se debería poder enviar únicamente estado, o combinar pacienteId con especialidad, o utilizar los seis filtros simultáneamente.

Nota: aplicar Spring Data JPA **Specifications**.

2. Una consulta que devuelva un resumen de citas por médico, con filtros opcionales por especialidad, estado y rango de fechas. Deberá devolver un DTO (MedicoResumenDTO).

Qué se debe obtener por cada médico:

- Su ID, nombre y especialidad.
- La cantidad de citas que coinciden con los filtros.
- Cuántas fueron atendidas.
- Cuántas fueron canceladas.

Sugerencia del DTO:

```
  public record MedicoResumenDTO(
    Long medicoId,
    String nombre,
    String especialidad,
    Long totalCitas,
    Long citasAtendidas,
    Long citasCanceladas
  ) {}
```

Nota: Utilizar filtros avanzados con @Query. Se anexa el fragmento en el archivo **~recortes.txt**.

Implemente el código restante y pruebe los endpoints

```
  GET /api/citas/resumen-medicos
  GET /api/citas/resumen-medicos?especialidad=Cardiología
  GET /api/citas/resumen-medicos?estado=PROGRAMADA
```

La respuesta debe tener una estructura similar a esta (ejemplo):

```
[
  {
    "medicoId": 101,
    "nombre": "Juan José Gómez",
    "especialidad": "Médico General",
    "totalCitas": 5,
    "citasAtendidas": 2,
    "citasCanceladas": 1
  }
  ...
]
```

3. Implementar un esquema completo de seguridad con cuatro perfiles, reglas de acceso por endpoint y una restricción de acceso a los datos según el médico autenticado.

Perfiles

Perfil	Responsabilidad
ROLE_ADMIN	Administración completa del sistema.
ROLE_MEDICO	Consultar citas propias y gestionar su atención.
ROLE_RECEPCIONISTA	Gestionar pacientes y programar citas.
ROLE_AUDITOR	Consultar información e informes, sin modificar datos.

Matriz de permisos

Recurso / operación	ADMIN	MÉDICO	RECEPCIONISTA	AUDITOR
GET /api/medicos	Sí	Sí	Sí	Sí
POST /api/medicos	Sí	No	No	No
GET /api/medicos/{id}	Sí	Sí*	Sí	Sí
PUT /api/medicos/{id}	Sí	No	No	No
DELETE /api/medicos/{id}	Sí	No	No	No
GET /api/pacientes	Sí	Sí	Sí	Sí
POST /api/pacientes	Sí	No	Sí	No
GET /api/pacientes/{id}	Sí	Sí*	Sí	Sí
PUT /api/pacientes/{id}	Sí	No	Sí	No
DELETE /api/pacientes/{id}	Sí	No	No	No
GET /api/citas	Sí	Sí*	Sí	Sí
POST /api/citas	Sí	No	Sí	No
PUT /api/citas/{id}	Sí	No	Sí	No
PATCH /api/citas/{id}/estado	Sí	Sí*	No	No
DELETE /api/citas/{id}	Sí	No	No	No
GET /api/citas/resumen-medicos	Sí	No	No	Sí

* Para el médico hay restricciones adicionales: solo puede consultar sus propias citas y los datos de pacientes relacionados con ellas. Solo puede cambiar el estado de una cita propia.

Requisitos técnicos

Implementar

- Entidades o modelo de usuario y roles, con usuarios almacenados en MySQL.
- POST /auth/login, que autentique las credenciales y devuelva un JWT.
- JwtService, responsable de generar y validar tokens.
- JwtAuthenticationFilter, que procese el encabezado Authorization.
- SecurityConfig, con sesiones STATELESS, configuración de rutas y filtro JWT.
- Contraseñas almacenadas con BCrypt, nunca en texto plano.
- Respuestas adecuadas: 401 Unauthorized para solicitudes sin autenticación válida y 403 Forbidden para usuarios autenticados sin permisos.
- Autorización por registro para impedir que un médico consulte o modifique citas de otros médicos.
- Pruebas con tokens válidos, inválidos, expirados y roles diferentes.

Orden recomendado de implementación

Para que el ejercicio sea más fácil/manejable, lo dividiría en cuatro pasos:

1. **Modelo de seguridad**: usuarios, roles y carga de usuarios desde la base de datos.
2. **Autenticación JWT**: login, AuthenticationManager, JwtService y filtro.
3. **Autorización por perfiles**: configuración de rutas y restricciones por método.
4. **Autorización por recurso y pruebas**: validar que un médico no pueda acceder a citas ajenas, incluso manipulando el ID de la URL.

Entregable: proyecto **medicitas** cargado a Teams con los 3 requerimientos implementados